

Hinweise und Tipps zu Aufgabenblatt 1

Sie implementieren eine **Agentensimulation** in einer Programmiersprache Ihrer Wahl. Geschwindigkeit des Codes spielt keine Rolle, auf Parallelität bzw. Multithreading können Sie gerne verzichten, denn das ist schwierig zu kontrollieren. Sie brauchen auch keine graphischen User Interfaces umsetzen, wichtig ist einzig die korrekte Funktionalität. Eventuell ist für das Debugging aber eine einfache graphische Ausgabe hilfreich. In jedem Fall müssen Sie den Verlauf und die Ergebnisse mehrerer Simulationen protokollieren. Hierfür dürfen Sie gerne Frameworks wie Log4J verwenden. Auch andere Bibliotheken dürfen Sie nutzen. Falls diese Bibliotheken bereits Agentenfunktionalitäten beinhalten, holen Sie sich vorher das Einverständnis des Lehrveranstalters ab.

Die nachfolgenden Informationen können Sie für Ihre Implementierung verwenden. Sie können für die nachfolgenden Aufgabenblätter auf dieser Gridwelt aufbauen, müssen das aber nicht. In den späteren Aufgabenblättern spielen Items und Entfernungen (= Kosten) eine Rolle, aber nicht mehr das Bewegen im Grid und das Aufnehmen und Ablegen von Items, sondern das Verhandeln mit anderen Agenten hinsichtlich des Zugriffs auf die Items.

Gridwelt

Teilautonome Agenten wirken in einer **zweidimensionalen Kachelwelt**, die von einem **Manager** kontrolliert wird. Auf einigen Feldern befinden sich **Items** verschiedener Typen in unterschiedlichen Mengen. Aufgabe der **Agenten** ist das Transportieren dieser Items. Die Aktionen der Agenten beschränkt sich dabei auf das Bewegen auf ein Nachbarfeld (4- oder 8-Nachbarschaft) und auf das Beladen und Entladen von Items.

Die Gridwelt ist eine $n \times m$ -Matrix von Feldern. Jedes Feld hat eine Kapazität, die angibt, wieviele Agenten sich auf dem Feld aufhalten können. Felder mit Kapazität 0 können nicht betreten werden, sie stellen Hindernisse dar. Agenten können die Gridwelt nicht verlassen.

Auf einem Feld können sich verschiedene Items befinden. Diese Items können von Agenten aufgenommen, wegtransportiert und wieder abgelegt werden. Jedes Item hat einen Typ und eine Mengenangabe. Die maximale Menge von Items auf einem Feld ist unbegrenzt. Das Auftanken mit Energie wird ebenfalls über Items vom Typ *Energie* umgesetzt.

Der Ablauf in der Gridworld ist getaktet: In einem Zeittakt kann ein Agent exakt eine Aktion durchführen. Diese Aktion verbraucht Energie.

Manager

Der Manager (wir sprechen hier nicht von einem Agenten) kontrolliert die Gridwelt. Er hat vollständiges Wissen über das Grid und den Zustand aller Agenten. Er kontrolliert die Zeit und die Umgebung der Agenten. Er ...

- startet und terminiert eine Simulation aus einer Modelldatei;
- startet die Agenten;
- kontrolliert die getaktete Zeit;
- stellt allen Agenten ihre Wahrnehmungen zu;
- kontrolliert den Zustand aller Agenten (auch Energieverbrauch und Aufladen von Energie);
- ermöglicht jedem Agenten in einem Zeittakt zu handeln;
- führt die Handlungen aus und protokolliert die Änderungen der Umgebung;

- benachrichtigt die Agenten über den Erfolg ihrer Aktionen.

Wie der Manager mit fehlerhaften Aktionen umgeht, ist Ihre Entscheidung. Entscheidend ist, dass die Gridwelt konsistent bleibt: Agenten können nur handeln, wenn ihr Energielevel > 0 ist und verändern die Welt nur bei erfolgreichen Aktionen (hier: Nahrungsaufnahme, -ablage und Laufen auf ein Nichthindernisfeld)(Dies ist insbesondere auch relevant für die nachfolgenden Aufgabenblätter, wenn Agenten falsch kommunizieren oder fehlerhafte Pläne erstellen)

Agenten

Ein Agent befindet sich immer auf einem eindeutigen Feld. Jeder Agent hat einen Energievorrat. Ohne Energie kann der Agent nicht handeln (er stirbt). (Für spätere Aufgabenblätter außerdem: Er hat eine maximale Ladekapazität für zu transportierende Items).

Für einen Agenten läuft jeder Zeittakt (Schritt) in 3 Phasen ab:

1. Sense: Der Agent erhält vom Manager Informationen. Dabei handelt es sich um Rückmeldungen über den Erfolg vergangener Aktionen (z.B. gescheiterte Bewegungsaktion) und um Wahrnehmungen im aktuellen Zustand (Pheromone und Items auf dem aktuellen Feld, ggfs. auch die Beladung des Agenten). Die Wahrnehmungen aktualisieren das Weltmodell des Agenten (Sie können das direkt durch den Manager erfolgen lassen, indem er den internen Zustand des Agenten manipuliert, oder der Agent implementiert eine entsprechende Funktion).
2. Reason: Der Agent entscheidet, was er als nächstes tun wird. Bei einem reaktiven Agenten (Aufgabenblatt 1) ist die Entscheidungsprozedur einfach (probabilistisch reaktiv).
3. Act: Der Agent sendet die nächste Aktion an den Manager. (Erfolgreiche) Aktionen verbrauchen Energie. Nur das Auftanken von Energie (Aufnehmen von Items der Kategorie *Energie*) verbraucht keine Energie, sondern füllt den Energievorrat

Simulationen

Simulationen sollen reproduzierbar sein¹. Alle relevanten Parameter und Informationen zur Startkonfiguration (Zahl der Agenten, Initialisierung der Agenten, Initialisierung der Felder, ...) sollen aus einer Datei (z.B. JSON) eingelesen werden: Topologie der Gridwelt, Verteilung der Ressourcen in der Gridwelt, Agenten (Name, Typ, Aufenthaltsort, Eigenschaften), weitere Parameter (z.B. Energieverbrauch pro Aktion, Dauer der Simulation).

Arbeitsweise des Managers (Vorschlag)

Innerhalb eines Zeittakts triggert der Manager jeden (lebenden) Agenten, sodass dieser seinen Sense-Reason-Act-Mechanismus ausführen kann. Weil wir auf echte Parallelität verzichten, kann dies sequenziell geschehen.

Konflikte löst der Manager selbständig auf. Beispielsweise "first come, first serve", wenn mehrere Agenten auf dieselbe Ressource zugreifen wollen.

¹ Deterministisch, mit Ausnahme von Situationen, wo ein Zufallsgenerator verwendet wird.

Illegale Aktionen werden abgewiesen: das Verlassen des Grids, das Aufnehmen nicht existenter Items, das Beladen jenseits der Agentenkapazität, und auch Aktionen, die mehr Energie verbrauchen als dem Agenten zur Verfügung steht.

Eine mögliche (einfache) Umsetzung wäre:

- Der Manager verwaltet Mengen von *Actions* und *Messages* und eine Liste von Indizes, die die einzelnen Agenten "adressieren".
- Im Rahmen des *Sense-Reason-Act*-Verarbeitungszyklus hängt ein Agent seine Aktion einfach an die *Actions* an und fügt (ab Aufgabenblatt 2) die zu versendenden Nachrichten in die *Messages* ein.
- Zu Beginn eines neuen Zeittaktes erfolgt die Zustandsaktualisierung, indem der Manager die *Actions* und *Messages* verarbeitet: In einer Schleife wird nacheinander jeder Agent "aktualisiert": Es wird die erste Aktion des Agenten auf Anwendbarkeit geprüft und dann (im Erfolgsfall) ausgeführt. Eine Erfolgsnachricht an Agent *i* wird in die Eingabequeue des Agenten *i* gelegt und der Zustand des Agenten manipuliert (z.B. Energielevel gesenkt). Danach durchsucht der Manager die *Messages* nach Broadcast-Nachrichten und direkt an den Agenten adressierte Nachrichten. Diese Nachrichtenmenge legt der Manager ebenfalls in die Eingabequeue des Agenten. Das Ergebnis einer Aktion und die versendeten Nachrichten werden dann noch für das Logging protokolliert.
Bei dieser Vorgehensweise wird maximal 1 Aktion pro Agent (nämlich die erste in den *Actions* ausgeführt).
- Als nächstes triggert der Manager in einer Schleife alle Agenten, damit diese ihren *Sense*, *Reason*, *Act*-Mechanismus ausführen. Innerhalb dieses Verarbeitungszyklus schreibt ein Agent dann wieder eine neue Aktion und ggfs. Nachrichten in die *Actions* und *Messages* des Managers.
- Anschließend wird der nächste Zeittakt gestartet...

Der offensichtliche Vorteil dieser Umsetzung liegt in der sequenziellen (deterministischen) Verarbeitung und dem damit verbundenen einfachen Debugging. Ein Nachteil ist, dass die fixe Reihenfolge der Agentenverarbeitung zu immer denselben Konfliktauflösungen führt: Wenn mehrere Agenten auf ein Feld laufen wollen, bekommt immer der 1. Agent das Vorrecht. Das mag unfair sein, ist aber akzeptabel.